

Планирование цепочек интенций поверх Forward–Backward представлений

Исследовательский отчёт

23 августа 2026 г.

Содержание

1	Постановка задачи	2
1.1	Что дано в исходном проекте	2
1.2	Почему одной подцели может быть мало	2
1.3	Начальная гипотеза	2
2	Математическое обобщение: многократное переключение	2
2.1	Одно переключение	2
2.2	Два переключения	2
2.3	Рекурсия для произвольного K	3
2.4	Switching advantage цепочки	3
3	Выбор архитектуры ViMixer	3
3.1	Какие варианты я рассматривал	3
3.2	Вдохновение Chain-of-Goals	4
3.3	Отличия предлагаемой модели	4
3.4	Offline supervision и loss	4
4	Эксперименты	5
4.1	Протокол	5
4.2	Основные результаты	5
5	Анализ результатов	5
5.1	Что важно вынести из результатов	5
6	Подробный разбор	5
6.1	Несоответствие между обучением и исполнением	5
6.2	Chain loss слабо наказывает один плохой переход	6
6.3	Фиксированная длина не соответствует сложности задачи	6
6.4	Что всё ещё нельзя утверждать	6
7	Вывод	6

1 Постановка задачи

1.1 Что дано в исходном проекте

В исходном проекте уже обучены три важных компонента. Первый компонент — forward representation $F(s, a, z)$, которая оценивает будущее поведение для интенции z . Второй — backward representation $B(s)$, переводящая состояние в то же латентное пространство. Третий — low-level политика $\pi_l(a \mid s, z)$, которая получает состояние и интенцию, а затем выдаёт действие.

Когда появляется новая reward-функция, она кодируется латентом z_r . После этого high-level actor решает, какую интенцию передать low-level политике. В изначальном решении актер выбирает одну интенцию. На следующем шаге задача решается заново уже из нового состояния.

1.2 Почему одной подцели может быть мало

Проблема текущего решения заключается в том, что актер не планирует свои действия наперед, а берет просто лучший ход в данный момент времени. Хотя в некоторых задачах, нужно совершить заведомо невыгодное действие, что например срезать путь. Такое случается на развилках или в тупиках.

1.3 Начальная гипотеза

Перед экспериментом я составил несколько гипотез относительно данной задачи:

1. если сеть предсказывает всю цепочку совместно, то первая подцель будет учитывать дальний маршрут и реже вести в тупик;
2. увеличение числа подцелей должно помогать на более длинных и неоднозначных задачах.

2 Математическое обобщение: многократное переключение

Этот раздел нужен, чтобы математически обосновать данное решение. И вывести формулу advantage. По сути он полностью основывается на статьях [1][2].

2.1 Одно переключение

Обозначим через $M_s^\mu(s')$ discounted successor measure политики μ , стартовой из s . Теорема 1 из [1] для политики $\pi_w \rightarrow \pi$, переключающейся в первом hitting time состояния w , имеет вид

$$M_s^{\pi_w \rightarrow \pi}(s') = M_s^{\pi_w}(s') + \frac{M_s^{\pi_w}(w)}{M_w^{\pi_w}(w)} (M_w^\pi(s') - M_w^{\pi_w}(s')). \quad (1)$$

Отношение

$$\rho(s, w; \pi_w) := \frac{M_s^{\pi_w}(w)}{M_w^{\pi_w}(w)}$$

2.2 Два переключения

Пусть сначала исполняется π_{w_1} , затем π_{w_2} и, наконец, π . Сначала применим (1) к хвосту цепочки:

$$M_{w_1}^{\pi_{w_2} \rightarrow \pi}(s') = M_{w_1}^{\pi_{w_2}}(s') + \rho_2 (M_{w_2}^\pi(s') - M_{w_2}^{\pi_{w_2}}(s')), \quad (2)$$

$$\rho_2 := \frac{M_{w_1}^{\pi_{w_2}}(w_2)}{M_{w_2}^{\pi_{w_2}}(w_2)}. \quad (3)$$

После этого хвост $\pi_{w_2} \rightarrow \pi$ рассматривается как единая политика, на которую происходит переключение в w_1 :

$$M_s^{\pi_{w_1} \rightarrow \pi_{w_2} \rightarrow \pi}(s') = M_s^{\pi_{w_1}}(s') + \rho_1 (M_{w_1}^{\pi_{w_2} \rightarrow \pi}(s') - M_{w_1}^{\pi_{w_1}}(s')), \quad (4)$$

где

$$\rho_1 = \frac{M_s^{\pi_{w_1}}(w_1)}{M_{w_1}^{\pi_{w_1}}(w_1)}.$$

Подстановка (2) в (4) даёт явную вложенную формулу для двух подцелей.

2.3 Рекурсия для произвольного K

Введём tail-policy

$$\nu_{K+1} := \pi, \quad \nu_i := \pi_{w_i} \longrightarrow \nu_{i+1}, \quad i = K, \dots, 1.$$

Тогда многократное переключение задаётся рекурсией

$$M_x^{\nu_i}(s') = M_x^{\pi_{w_i}}(s') + \frac{M_x^{\pi_{w_i}}(w_i)}{M_{w_i}^{\pi_{w_i}}(w_i)} \left(M_{w_i}^{\nu_{i+1}}(s') - M_{w_i}^{\pi_{w_i}}(s') \right). \quad (5)$$

Для первой фазы $x = s$, а для последующих хвостов стартовым состоянием является предыдущая подцель.

Чтобы записать развёрнутую форму, положим $w_0 = s$ и

$$\rho_i := \frac{M_{w_{i-1}}^{\pi_{w_i}}(w_i)}{M_{w_i}^{\pi_{w_i}}(w_i)}, \quad \pi_{w_{K+1}} := \pi.$$

Последовательная подстановка (5) даёт

$$M_s^{\nu_1}(s') = M_s^{\pi_{w_1}}(s') + \sum_{i=1}^K \left(\prod_{j=1}^i \rho_j \right) \left(M_{w_i}^{\pi_{w_{i+1}}}(s') - M_{w_i}^{\pi_{w_i}}(s') \right). \quad (6)$$

2.4 Switching advantage цепочки

Для reward-функции r определим advantage всей составной политики относительно глобальной π :

$$A_s^{\nu_1}(r) := \langle M_s^{\nu_1} - M_s^{\pi}, r \rangle = \sum_{s'} M_s^{\nu_1}(s') r(s') - V^{\pi}(s; r). \quad (7)$$

Интегрируя (6) по r , получаем

$$\begin{aligned} A_s^{\nu_1}(r) &= V^{\pi_{w_1}}(s; r) - V^{\pi}(s; r) \\ &+ \sum_{i=1}^K \left(\prod_{j=1}^i \rho_j \right) (V^{\pi_{w_{i+1}}}(w_i; r) - V^{\pi_{w_i}}(w_i; r)). \end{aligned} \quad (8)$$

Это выражение оценивает цепочку как единый план и показывает, что локальная полезность w_i зависит от всего policy tail после неё.

Это и есть теоретическая мотивация генерировать последовательность подцелей. High-level политика должна выбирать не самую хорошую точку прямо сейчас, а совместимый набор точек, у которого полезна вся последовательность.

3 Выбор архитектуры ViMixer

3.1 Какие варианты я рассматривал

До реализации я рассматривал три способа получить последовательность интенций. Первый вариант — авторегрессионная модель, которая выдаёт z_1 , затем z_2 с учётом z_1 и так далее. Её преимущество в том, что мы можем генерировать произвольное количество связанных подцелей.

Проблема заключалась в том, что такая система способна накапливать ошибку. Если первая подцель выбрана неправильно, то последующие будут идти по этому неправильному пути.

Второй вариант — один общий латент z_{plan} , который кодирует маршрут целиком, и decoder, разворачивающий его в список. Это было интересной идеей, но я побоялся, что потеряю слишком много деталей. А также, что такое решение будет склонно к усреднению разных маршрутов.

Третий вариант — основывается на статье [2], где представлено решение по предсказанию последовательности подцелей Mixer, который от итоговой цели прокладывает маршрут подцелей до текущего положения, чтобы сделать последовательность. В статье отмечается, что есть вектор развития этого решения, а именно сделать его двунаправленным. Поскольку Mixer, если задуматься, просто решает ту же самую задачу, только задом наперед. При этом я заранее понимал ограничение моего решения, а именно строго выбранное количество подцелей на задачу. Но я решил использовать это, чтобы параллельно проверить гипотезу про количество подцелей.

3.2 Вдохновение Chain-of-Goals

Chain-of-Goals Hierarchical Policy (CoGHP) [2] рассматривает латентные подцели как reasoning токены. CoGHP использует MLP-Mixer, авторегрессионно генерирует подцели от самой дальней к ближайшей и применяет lower-triangular causal mixer. Благодаря этому ближайшая подцель учитывает уже созданные дальние подцели.

3.3 Отличия предлагаемой модели

Вместо causal autoregressive generation используется bidirectional parallel generation:

$$[e_s, e_g, \tilde{z}_1, \dots, \tilde{z}_N] \mapsto [z_1, \dots, z_N].$$

Здесь e_s и e_g — MLP-кодировки состояния и goal latent, а $\tilde{z}_i$ — обучаемые slot tokens. Полная mixing matrix позволяет каждому slot обмениваться информацией со всеми остальными.

Выбор bidirectional Mixer сделан по следующим причинам:

- весь план получается за один forward pass;
- отсутствует накопление ошибок;
- каждая подцель сохраняет знание о финальной задаче;

3.4 Offline supervision и loss

Для состояния с индексом t и будущей цели g из той же траектории waypoint выбираются как

$$t_i = t + \left\lceil (g - t) \frac{i}{N + 1} \right\rceil, \quad z_i = \text{normalize}(B(s_{t_i})).$$

Маски удаляют повторные переходы на коротких trajectory fragments.

Такой способ разметки очень простой. Но у него есть цена. Равномерное расстояние по индексам траектории не означает равномерное расстояние в лабиринте: между двумя соседними подцелями может оказаться поворот, длинная стена или развилка. Кроме того, в offline датасете одна и та же пара областей может быть соединена разными маршрутами.

Итоговый objective:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E} \left[e^{\text{clip}(\alpha \hat{A}_{\text{chain}}, 5)} \frac{1}{N} \sum_i \log \pi_h(z_i \mid s, z_r) \right] + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}. \quad (9)$$

Оптимизируется только high-level actor. Параметры F , B и π_l заморожены.

4 Эксперименты

4.1 Протокол

- Среда: `ogbench-antmaze-medium-navigate-v0`.
- Baseline: предоставленный single-intention checkpoint.
- BiMixer $N = 3$: checkpoint 400k; $N = 4, 5$: checkpoint 500k.
- Для всех BiMixer использован training seed 0.
- Zero-shot inference: 100 000 offline samples на задачу.
- Evaluation seeds: 0, 1, 2, 3, 4.
- 100 эпизодов на каждую из пяти задач и каждый seed.

4.2 Основные результаты

Таблица 1: Success rate: mean $\pm$ standard deviation по evaluation seeds.

Метод	Task 1	Task 2	Task 3	Task 4	Task 5	Overall
Baseline	0.816 ± 0.057	0.838 ± 0.026	0.756 ± 0.021	0.718 ± 0.018	0.822 ± 0.065	0.790 ± 0.023
BiMixer, $N = 3$	0.828 ± 0.047	0.870 ± 0.024	0.808 ± 0.037	0.596 ± 0.073	0.902 ± 0.036	0.801 ± 0.020
BiMixer, $N = 4$	0.754 ± 0.036	0.880 ± 0.019	0.698 ± 0.036	0.566 ± 0.030	0.848 ± 0.026	0.749 ± 0.014
BiMixer, $N = 5$	0.754 ± 0.026	0.866 ± 0.015	0.664 ± 0.059	0.572 ± 0.059	0.764 ± 0.025	0.724 ± 0.023

Разницы относительно baseline приведены в таблице 2.

Таблица 2: Разница success rate относительно single-intention baseline.

Метод	Task 1	Task 2	Task 3	Task 4	Task 5	Overall
BiMixer, $N = 3$	+0.012	+0.032	+0.052	−0.122	+0.080	+0.0108
BiMixer, $N = 4$	−0.062	+0.042	−0.058	−0.152	+0.026	−0.0408
BiMixer, $N = 5$	−0.062	+0.028	−0.092	−0.146	−0.058	−0.0660

5 Анализ результатов

5.1 Что важно вынести из результатов

Во первых — сеть научилась выдавать приемлимый маршрут подцелей. Этого видно по метрикам, а также по графикам визуализации, которые лежат на диске.

Во вторых — сильного преимущества над baseline список подцелей не дали.

В третьих — увеличение количества подцелей не привело к увеличению метрик.

6 Подробный разбор

6.1 Несоответствие между обучением и исполнением

Это наиболее надёжно установленная причина. Loss оптимизирует likelihood всей последовательности, а action зависит непосредственно только от z_1 . Поэтому хороший z_4 или z_5 полезен лишь косвенно. У модели нет требования, что среда действительно исполнит последующие элементы.

В результате во время обучения и инференса система решает две разные задачи. При $N = 3$ эти цели частично совпали. При $N = 4, 5$ дополнительные задачи могли начать отвлекаться от самого важного выхода z_1 .

6.2 Chain loss слабо наказывает один плохой переход

В коде используется среднее локальных advantage. Среднее допускает компенсацию: сильные переходы могут скрыть один слабый.

Проблема становится заметнее при росте N . В цепочке из трёх элементов один плохой переход составляет треть плана. В цепочке из пяти его вклад в среднее меньше, хотя для реального исполнения он всё равно может полностью разрушить маршрут.

6.3 Фиксированная длина не соответствует сложности задачи

BiMixer всегда выдаёт ровно N slot. Но разным состояниям требуется разное число решений: в прямом коридоре достаточно одной цели, а перед несколькими развилками может понадобиться длинный план. Фиксированный N заставляет простые примеры искусственно заполнять все позиции и не умеет сказать, что план уже закончен.

6.4 Что всё ещё нельзя утверждать

Из результатов нельзя сделать вывод, что любая multi-subgoal политика хуже single-intention. Проверена одна архитектура, один способ разметки, один loss и один training seed на N . Также нельзя уверенно сказать, что $N = 3$ действительно лучше baseline в среднем по обучению.

Зато можно утверждать более узкие вещи:

- текущий BiMixer с $N = 3$ конкурентоспособен с baseline;
- простое увеличение N в текущем датасете и execution ухудшает overall;

7 Вывод

Изначальная практическая цель состояла в том, чтобы убедительно превзойти single-intention baseline. Этой цели текущая версия не достигла. $N = 3$ показал небольшой выигрыш, но общий доверительный интервал пересекает ноль. $N = 4$ и $N = 5$ оказались хуже. Поэтому было бы неправильно представлять результат как успешный новый sota.

Наиболее обоснованный следующий шаг — научиться выводить адаптивное количество подцелей и научить систему следовать своему плану.

Список литературы

- [1] Stojanovic and Proutiere. *Switching Successor Measures for Hierarchical Zero-Shot Reinforcement Learning*. 2026.
- [2] *Chain-of-Goals Hierarchical Policy for Long-Horizon Offline Goal-Conditioned Reinforcement Learning*. Manuscript used as architectural motivation.
- [3] Touati and Ollivier. *Learning One Representation to Optimize All Rewards*. 2021.
- [4] Park et al. *OGBench: Benchmarking Offline Goal-Conditioned RL*. 2025.